

P2 AED2

Daniel Pozas Ruiz (DNI: 26548389H) y Pedro Blaya Bascual (DNI: 48853636A)

Héctor Muñoz Ortiz. AED II 25-26

Grupo PCEO; subgrupo 1.3 (9.1)

Índice

1. Backtracking	2
1.1. Enunciado	2
1.2. Diseño	4
1.3. Análisis teórico	6
1.4. Implementación	7
1.5. Envíos	8
1.6. Análisis experimental	8
1.7. Contraste	10
2. Avance rápido	11
2.1. Enunciado	11
2.2. Diseño	12
2.3. Análisis teórico	15
2.4. Implementación	16
2.5. Envíos	17
2.6. Análisis experimental	17
2.6.1. Optimalidad de la aproximación	17
2.7. Contraste	18
3. Uso de la IA	20
4. Conclusiones	21
4.1. IMPORTANTE	21

Capítulo 1

Backtracking

1.1. Enunciado

Nos ha tocado hacer el problema "A_Ba - Asignación de tareas a trabajadores" para la sección de backtracking, a continuación el enunciado:

- **Contexto:**

En una empresa se dispone de varios trabajadores para realizar una serie de trabajos. Cada trabajador tiene una cierta capacidad de realizar trabajos y puede que haya trabajos que no sabe resolver. Vuestro trabajo consiste en decidir los trabajos a realizar por cada trabajador teniendo en cuenta su capacidad máxima de trabajo y los trabajos que no sabe cómo resolver. Se pretende maximizar el beneficio en la realización de los trabajos asignados, pues la habilidad en la realización de los trabajos por los trabajadores varía.

- **El problema:**

Tenemos nt trabajos que hay que asignar a nw trabajadores. Cada trabajador i tiene una capacidad $c(i)$, que indica el número máximo de tareas que se le pueden asignar. Si el trabajador i realiza el trabajo j , se obtiene un beneficio representado por una entrada de una tabla $b(i,j)$. Algunos trabajadores no saben resolver algunas tareas, lo que se representa en la tabla con $b(i,j)=0$. Se trata de obtener de todas las posibles asignaciones de trabajos a trabajadores que cumplen las restricciones (no se asigna un trabajo a un trabajador si no lo sabe hacer, y el número de trabajos asignado a un trabajador no excede su capacidad) la que maximiza el beneficio. El beneficio de una asignación es la suma de los beneficios de realización de los nt trabajos por los trabajadores a los que se asignan. Solo son soluciones válidas aquellas en que se asignan todos los trabajos. Cuando no haya solución válida la respuesta será 0.

- **Entrada:**

La primera línea de la entrada contiene un entero, T , que indica el número de casos de prueba. Cada caso de prueba contiene una primera línea con dos números con los valores de nw y nt . A continuación hay nw líneas, cada una con nt enteros, para representar la tabla b . Y a continuación una línea con nw enteros, para representar las capacidades.

- **Salida:**

Para cada caso de prueba, la salida es una línea con el máximo beneficio obtenido.

- **Ejemplo de entrada:**

```

4
5 5
0 2 0 0 0
0 0 0 2 0
0 0 1 0 0
0 0 0 0 0
0 0 0 0 1
1 2 2 2 1
8 6
4 0 0 0 0 0
4 5 0 0 0 0
0 4 0 0 0 0
2 0 0 3 2 0
0 0 0 4 2 0
5 5 2 0 2 3
0 0 2 2 2 2
0 2 0 0 2 5
1 1 1 1 1 1 1 1
6 8
4 0 0 0 0 0 4 5
0 0 0 0 0 4 0 0
0 0 2 0 0 3 2 0
0 0 0 4 2 0 5 5
2 0 2 3 0 0 2 2
2 2 0 2 0 0 2 5
1 1 1 1 1 1
6 8
4 0 0 0 0 0 4 5
0 0 0 0 0 4 0 0
0 0 2 0 0 3 2 0
0 0 0 4 2 0 5 5
2 0 2 3 0 0 2 2
2 2 0 2 0 0 2 5
2 2 2 2 2 2

```

- **Ejemplo de salida:**

```

0
23
0
27

```

1.2. Diseño

En este apartado discutiremos las cuestiones de diseño. Debido a que se trata de un problema en el que debemos de maximizar el beneficio obtenido cumpliendo una serie de restricciones y limitaciones aplicaremos el esquema iterativo de optimización. Además, añadiremos una serie de variables auxiliares para mejorar la eficiencia y evitar cálculos innecesarios:

- **Beneficio actual:** suma del beneficio acumulado por las tareas asignadas hasta el nivel actual.
- **Asignación por trabajadores:** vector que registra el número de trabajos asignados por trabajador para determinar si se le pueden asignar más tareas.
- **Libres:** registra el número total de tareas asignables entre todos los trabajadores para determinar si existen más trabajadores a los que asignar tareas sin tener que recorrer en cada nodo el vector anterior.

Usaremos un árbol k -ario siendo k el número de trabajadores con un número de niveles igual al número de tareas. Para la poda primero implementaremos una función para determinar si en el estado actual se podría superar al beneficio máximo almacenado mediante el cálculo de cotas superiores. Sea B la matriz de beneficios y C la matriz de capacidades de cada trabajador y s una solución cualquiera.

Algorithm 1 Función para calcular la cota superior

```

1: function cotaSuperior(nivel, beneficio, trabajosAsignados)
2:   cota  $\leftarrow$  beneficio
3:   for  $j \leftarrow nivel + 1$  hasta numTareas do
4:     maxB  $\leftarrow$  0
5:     for  $i \leftarrow 0$  hasta numTrabajadores do
6:       if trabajosAsignados[ $i$ ] <  $C[i]$  AND  $B[i][j] > maxB$  then
7:         maxB  $\leftarrow B[i][j]$ 
8:       end if
9:     end for
10:    cota  $\leftarrow cota + maxB$ 
11:  end for
12:  return cota
13: end function

```

Algorithm 2 Función criterio para la poda

```

1: function criterio(nivel,  $s$ , beneficioActual, beneficioMaximo, trabajosAsignados)
2:  return  $nivel < numTareas - 1$  AND
3:   $trabajosAsignados[s[nivel]] \leq C[s[nivel]]$  AND
4:   $B[nivel][s[nivel]] \neq 0$  AND NOT
5:   $cotaSuperior(nivel, beneficioActual, trabajadoresAsignados) \leq$ 
6:  beneficioMaximo
7: end function

```

El algoritmo 2 implementa, aparte de la poda por función objetivo usando el algoritmo 1, otras podas, como la poda por profundidad (primera comprobación, línea 2); la poda por capacidad (segunda comparación, línea 3) y la poda por beneficio (tercera comparación, línea 4), siguiendo los criterios que establece el enunciado.

A continuación se diseñan las funciones para implementar el esquema seguido, totalmente análogo al visto en clase. Usaremos, de nuevo, la nomenclatura vista en clase para que sea más fácil relacionar cada parte del código con la teoría.

Algorithm 3 Función para generar un nuevo componente de la solución

```

1: function generar(nivel, s, beneficio, trabajosAsignados)
2:   s[nivel]  $\leftarrow$  s[nivel] + 1
3:   if nivel = 0 then
4:     beneficio  $\leftarrow$  beneficio + B[nivel][s[nivel]]
5:     trabajosAsignados[s[nivel]]  $\leftarrow$  trabajosAsignados[s[nivel]] + 1
6:     libres  $\leftarrow$  libres - 1
7:   else
8:     beneficio  $\leftarrow$  beneficio + B[nivel][s[nivel]]
9:     trabajosAsignados[s[nivel]]  $\leftarrow$  trabajosAsignados[s[nivel]] + 1
10:    beneficio  $\leftarrow$  beneficio - B[nivel - 1][s[nivel]]
11:    trabajosAsignados[s[nivel - 1]]  $\leftarrow$  trabajosAsignados[s[nivel - 1]] - 1
12:   end if
13: end function

```

Algorithm 4 Función para retroceder una componente en la solución

```

1: function retroceder(nivel, s, beneficio, trabajosAsignados)
2:   beneficio  $\leftarrow$  beneficio - B[nivel][s[nivel]]
3:   trabajosAsignados[s[nivel]]  $\leftarrow$  trabajosAsignados[s[nivel]] - 1
4:   libres  $\leftarrow$  libres + 1
5:   s[nivel]  $\leftarrow$  invalid
6:   nivel  $\leftarrow$  nivel - 1
7: end function

```

Algorithm 5 Función para determinar si *s* es solución válida

```

1: function solucin(nivel, s, trabajosAsignados)
2:   return nivel = numTareas - 1 AND
3:   trabajosAsignados[s[nivel]]  $\leq$  C[s[nivel]] AND
4:   B[nivel][s[nivel]]  $\neq$  0
5: end function

```

Nota: para este último algoritmo revisamos solo si la solución está completa y que la última componente está bien elegida, el resto de componentes se encarga el algoritmo 2 de verificarlas.

Algorithm 6 Función para determinar si quedan hermanos en un nivel del árbol

```

1: function masHermanos(nivel, s)
2:   return  $s[nivel] < numTrabajadores - 1$  AND libres  $\neq 0$ 
3: end function

```

Ahora, juntando todas las funciones que hemos ido diseñando, construimos el esquema principal. Tenemos en cuenta que las variables auxiliares de optimización están ya inicializadas:

Algorithm 7 Esquema BT optimización con poda FO

```

1: solucionMaximal  $\leftarrow$  solucionVacía
2: beneficioMaximo  $\leftarrow$  0
3: solucionActual  $\leftarrow$  solucionVacía
4: beneficioActual  $\leftarrow$  0
5: nivel  $\leftarrow$  0
6: while nivel  $\neq -1$  do
7:   generar(nivel, solucionActual, beneficioActual, trabajosAsignados)
8:   if solucion(nivel, solucionActual, trabajosAsignados) then
9:     if beneficioActual  $\geq$  beneficioMaximo then
10:      solucionMaximal  $\leftarrow$  solucionActual
11:      beneficioMaximo  $\leftarrow$  beneficioActual
12:    end if
13:  end if
14:  if criterio(nivel, solucionActual, beneficioActual, beneficioMaximo, trabajosAsignados)
15:    then
16:      nivel  $\leftarrow$  nivel + 1
17:    else
18:      while nivel  $\geq 0$  AND NOT masHermanos(nivel, solucionActual) do
19:        retroceder(nivel, solucionActual, beneficioActual, trabajosAsignados)
20:      end while
21:    end if
22:  end while

```

1.3. Análisis teórico

Dado que el análisis teórico del algoritmo tal y como lo hemos diseñado (poda con función objetivo) es complicado, vamos a intentar compararlo con un algoritmo que realiza una poda más simple (simplemente en el algoritmo 2, función *criterio*, no se realiza la última comparación):

En primer lugar, señalar que los algoritmos 3-6 son trivialmente $\Theta(1)$ y que el algoritmo 2 tendrá la misma complejidad que el algoritmo 1. Es fácil observar que, por tanto, la complejidad por nodo es la complejidad del algoritmo 2. El problema se reduce a hallar la complejidad del algoritmo 1, denotemos por nt, nw el número de tareas y trabajadores, respectivamente y por $t(nt, nw)$ el tiempo de ejecución del algoritmo 1:

$$t(nt, nw) = c_1 + (nt - nivel - 1) \cdot (c_2 + nw \cdot c_3 + c_4) \quad (1.1)$$

Es claro entonces que $t(nt, nw, nivel) \in \Theta((nt - nivel) \cdot nw)$ ya que:

$$\lim_{(nt-nivel) \cdot nw \rightarrow \infty} \frac{t(nt, nw)}{(nt - nivel) \cdot nw} = L \quad (1.2)$$

Es decir, $\Theta(nt \cdot nw)$ en el peor caso, con $nivel = 0$ y $\Theta(nw)$ en el mejor caso, con $nivel = nt - 1$ (recordemos que $0 \leq nivel \leq nt - 1$).

Hallar el caso promedio es intratable teóricamente pues nos es imposible determinar la probabilidad de encontrarse en determinado nivel del árbol sin cálculos experimentales, más aún sabiendo que usamos una poda muy bien optimizada y se producen muchos retrocesos. Tomemos pues el peor caso y demos una cota superior del algoritmo 7. Señalar que las funciones tales que su tiempo de ejecución sea $\Theta(1)$ están trivialmente acotadas superior e inferiormente por constantes. Teniendo en cuenta que el número de nodos es nw^{nt} (árbol con nt niveles de profundidad y nw hermanos) y denotando por $t_W(nt, nw)$ el tiempo de ejecución del código dentro del bucle while, es decir, el tiempo por nodo.

El tiempo del peor caso de BT (se recorren todos los nodos), vendrá dado por:

- **Versión sin poda FO:** tiempo por nodo $\Theta(1)$, pues t_W está acotada por tiempos constantes por arriba y por abajo (debido a que toda función $\Theta(1)$ también lo está), en total $\Theta(nw^{nt})$
- **Versión con poda FO:** debido a los múltiples factores que influyen en el tiempo de ejecución de la función *criterio* (poda prematura antes de llegar a ejecutar la función *cotaSuperior* o influencia del nivel del nodo en el tiempo de ejecución de esta última) daremos una cota superior del tiempo del peor caso de BT, sabiendo que la función *criterio* es $O(nt \cdot nw)$, lo que implica que $t_W \in O(nt \cdot nw)$, quedándonos en total con $O(nt \cdot nw^{nt+1})$. Puede parecer peor en perspectiva pero tenemos la esperanza de que se produzcan podas importantes y el tiempo promedio en este caso sea mejor que sin poda aunque eso habremos de determinarlo experimentalmente.

1.4. Implementación

- Fichero **bt.cpp**: Implementa, comentado y explicado, el backtracking con poda FO y sin poda FO (simplemente cambia una línea).
- Fichero **bt.h**: Cabecera de *bt.cpp* para poder usar más cómodamente las funciones a la hora de calcular los tiempos.
- Fichero **bt_mooshak.cpp**: Código enviado a Mooshak, pero que no se usa en el estudio estadístico debido a sus limitaciones.
- Fichero **tiempos_BT.cpp**: Se encarga de medir y comparar a detalle los tiempos de ejecución del algoritmo con poda FO y sin poda FO, almacenándolos en .csv.
- Fichero **grafica_poda.py**: a partir del .csv generado construye una gráfica comparando las mediciones para el algoritmo con y sin poda FO.
- Fichero **makefile**: Describe como compilar el proyecto.

Notas: el archivo Python que genera las gráficas puede parecer algo escueto en relación a lo desarrollado en la práctica 1 o directamente en AR, pero realmente solo necesitábamos una gráfica comparando las mediciones entre ambos algoritmos ya que es lo único que podemos desarrollar en el análisis experimental.

1.5. Envíos

#	Tiempo de Concurso	País	Equipo	Problema	Lenguaje	Resultado	Estado
381	917:27:04		g9_g9 08	A_Ba	C++	2 Accepted	final
380	917:23:05		g9_g9 08	A_Ba	C++	1 Time Limit Exceeded	final
379	917:17:47		g9_g9 08	A_Ba	C++	1 Time Limit Exceeded	final
378	917:06:50		g9_g9 08	A_Ba	C++	1 Time Limit Exceeded	final
377	916:13:56		g9_g9 08	A_Ba	C++	0 Wrong Answer	final

Figura 1.1: Listado de envíos

Hay una serie de envíos anteriores pero ninguno recordamos lo que eran, así que no podemos decir nada al respecto y no los hemos incluido. Usuario g9_08:

- Envío **377**: Primer envío, el algoritmo realizaba algunas comparaciones incorrectamente.
- Envíos **378, 379, 380**: El programa funcionaba correctamente pero era demasiado lento, tras varios intentos y optimizaciones caímos en usar una poda FO.
- Envío **381**: El último envío, tras implementar la poda FO Mooshak lo aceptó. Este el único envío aceptado por Mooshak y por tanto el que hay que **corregir**.

1.6. Análisis experimental

Este análisis experimental se basara en demostrar simplemente que el algoritmo con poda es mucho más eficiente que el algoritmo sin poda, pues es imposible entrar en el tema de las regresiones debido a que los algoritmos son tremendamente costosos y tardaríamos horas en generar las gráficas y, además, para el algoritmo con poda ni siquiera hemos sido capaces de encontrar un orden exacto.

El procedimiento que seguiremos es el siguiente: generaremos casos aleatorios de tamaño de entrada $nt_i = \text{randomInt}(6, 8)$ la razón de un rango tan pequeño es que queremos un tamaño de lo suficientemente significativo, el problema, es que al depender el algoritmo exponencialmente de nt_i si subimos de 8 entonces el tiempo de ejecución se dispara y, de nuevo, tardamos horas en sacar resultados significativos. $nw_i = \lceil \text{randomReal}(0,5, 1,5) \cdot nt_i \rceil$, es una manera que tenemos de que nw_i crezca arbitrariamente con nt_i , ponemos el multiplicador algo alto ya que el impacto de nw_i no es tan significativo como el nt_i . Después, compararemos el tiempo de ejecución del algoritmo con poda y sin poda, generando datos como mínimo, máximo, mediana o media del empeoramiento/mejoramiento; media, mediana y máximo de cuanto se acelera (speedup) o decelera (slowdown) de la ejecución y, por último, una tasa de casos en los que con poda mejora a sin poda:

Caso	nt	nw	T. Sin Poda	T. Con Poda	Dif. %	Ratio	Resultado
------	----	----	-------------	-------------	--------	-------	-----------

1	6	8	2.54588	0.033759	98.674	75.41	Mejor
2	7	8	17.3345	0.033014	99.809	525.06	Mejor
3	7	10	34.108	0.02126	99.937	1604.33	Mejor
4	7	4	0.003597	0.005461	-51.821	0.65	Peor
5	7	4	0.071077	0.002638	96.288	26.94	Mejor
6	8	10	266.421	0.090627	99.966	2939.75	Mejor
7	6	5	0.063643	0.002354	96.301	27.03	Mejor
8	6	4	0.009937	0.002702	72.808	3.67	Mejor
9	8	10	366.349	0.038039	99.989	9630.88	Mejor
10	7	4	0.025061	0.004279	82.925	5.85	Mejor
11	8	5	3.7547	0.007782	99.792	482.48	Mejor
12	7	10	50.5368	0.032433	99.935	1558.19	Mejor
13	7	7	5.84318	0.013091	99.776	446.35	Mejor
14	8	11	544.181	0.111612	99.979	4875.65	Mejor
15	8	4	0.073443	0.002105	97.133	34.88	Mejor
16	6	3	0.004525	0.001817	59.845	2.49	Mejor
17	6	9	3.36549	0.011925	99.645	282.22	Mejor
18	6	3	0.002345	0.001488	36.545	1.57	Mejor
19	8	10	494.788	0.036062	99.992	13720.5	Mejor
20	8	5	2.93174	0.007612	99.740	385.14	Mejor
21	7	5	0.326843	0.004711	98.558	69.37	Mejor
22	6	4	0.021117	0.002198	89.591	9.60	Mejor
23	8	11	635.398	0.04211	99.993	15089.0	Mejor
24	7	9	29.8043	0.015525	99.947	1919.76	Mejor
25	7	6	0.495399	0.007199	98.546	68.81	Mejor
26	7	7	3.69841	0.016611	99.550	222.64	Mejor
27	7	6	2.22792	0.006875	99.691	324.06	Mejor
28	6	3	0.00532	0.000994	81.315	5.35	Mejor
29	8	11	910.934	0.036032	99.996	25281.3	Mejor
30	7	10	68.4201	0.020244	99.970	3379.77	Mejor

RESUMEN

Total de casos: 50

Con poda MEJOR: 49 (98 %)

Con poda PEOR: 1 (2 %)

Empate: 0 (0 %)

Cuando CON PODA es MEJOR:

Mejora porcentual:

media = 94.0626 %

mediana = 99.1968 %

min = 36.5458 %

max = 99.9962 %

Speedup (sin/con):

media = 2379.53x

mediana = 124.509x
 max = 26248.1x

Cuando CON PODA es PEOR:

Empeoramiento porcentual:

media = 51.821 %
 mediana = 51.821 %
 min = 51.821 %
 max = 51.821 %

Slowdown (con/sin):

media = 1.51821x
 mediana = 1.51821x
 max = 1.51821x

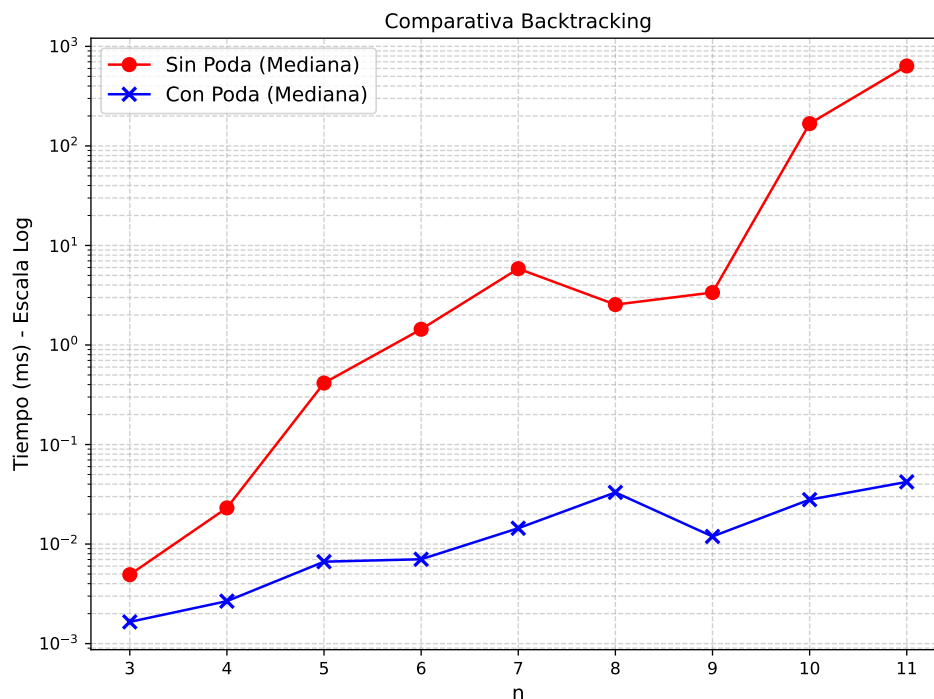


Figura 1.2: $n := nw$ (nt varía muy poco)

Con esto queda demostrado que, como sospechábamos, la versión poda es mucho mejor que la versión sin poda.

1.7. Contraste

Hemos decidido no incluir esta sección en backtracking, razones explicadas **con todo lujo de detalle** en la sección 4.1 (Conclusiones - IMPORTANTE), página 21, último párrafo. Rogamos no se nos penalice, pues creemos tener argumentos de peso.

Capítulo 2

Avance rápido

2.1. Enunciado

Nos ha tocado hacer el problema C_AR de avance rápido, a continuación el enunciado:

- **Contexto:**

De una población se pretende seleccionar una subpoblación, y se quiere que la población sea diversa, por lo que se seleccionan los individuos más distantes entre sí.

- **El problema:**

Tenemos n elementos de los que hay que seleccionar m . Entre cada dos elementos i y j hay una distancia $d(i,j)$. De todos los subconjuntos de m elementos queremos obtener el que da la mayor suma de distancias entre elementos en el subconjunto: $\max_S \sum_{i,j \in S} d(i,j)$

- **Entrada:**

La primera línea de la entrada contiene un entero, T , que indica el número de casos de prueba. Cada caso de prueba contiene una primera línea con dos números con los valores de n y m . A continuación hay n líneas, cada una con n enteros, para representar las distancias d . La distancia de un elemento a sí mismo será siempre cero, y las distancias entre dos elementos dados pueden variar dependiendo del sentido.

- **Salida:**

Para cada caso de prueba la salida consta de dos líneas. La primera línea contiene la máxima diversidad obtenida, y la segunda línea representa el subconjunto con el que se obtiene la máxima diversidad, con n 0 y 1, con 0 para indicar un elemento que no se incluye en el subconjunto y 1 indicando que el elemento se incluye.

- **Ejemplo de entrada:**

```
3
4 2
0 3 2 4
2 0 4 5
```

```

2 1 0 4
2 3 2 0
4 3
0 3 2 4
2 0 4 5
2 1 0 4
2 3 2 0
6 3
0 5 4 2 5 3
4 0 1 1 3 3
2 5 0 3 0 4
3 3 4 0 4 7
2 5 7 5 0 6
4 8 5 6 8 0

```

■ **Ejemplo de salida:**

```

8
0 1 0 1
19
1 1 0 1
36
0 0 0 1 1 1

```

2.2. Diseño

En este apartado se discutirán las cuestiones de diseño. Para resolver el problema seguiremos el esquema voraz, la decisión principal será tomar, localmente el par no elegido que mayor diversidad (es decir, dados i, j mayor $d(i, j) + d(j, i)$) aporte. Adelantamos que para este problema esta decisión no nos proporciona un algoritmo voraz, aunque se trata de una buena aproximación (lo comprobaremos experimentalmente). Tras diseñar los algoritmos pertinentes justificaremos con un ejemplo lo anterior y, además, trataremos de justificar que no existe un algoritmo voraz exacto que resuelva este problema.

En primer lugar, diseñaremos el equivalente en nuestro programa a la función *insertar* de los esquemas vistos en teoría. Definiremos el tipo de datos *Solucion* como el par $(beneficio, listaElegidos)$, es decir, el beneficio que aporta la elección de los individuos de la lista.

Algorithm 8 Función para actualizar el beneficio de la solución final

```

1: function actualizarBeneficios(escogidos, beneficio, indice)
2:   for  $i$  en escogidos do
3:      $beneficio \leftarrow beneficio + distancias[indice][escogidos[i]]$ 
4:      $beneficio \leftarrow beneficio + distancias[escogidos[i]][indice]$ 
5:   end for
6:    $add(escogidos, indice)$ 
7: end function

```

Ahora, diseñaremos el algoritmo principal, tendremos en cuenta que *poblacion* es una

matriz bidimensional que almacena la suma $d(i, j) + d(j, i)$ de cada uno de los pares. Con *ordenar*(*poblacion*[*i*]) nos referimos a ordenar la lista de pares de la fila *i* por beneficio aportado:

Algorithm 9 Algoritmo AR principal

```

1: function avanceRapido(n, m)
2:   for i hasta n do
3:     ordenar(poblacion[i])
4:   end for
5:   elegidos  $\leftarrow$  vectorDeNCeros
6:   indices  $\leftarrow$  vectorDeNCeros
7:   subpoblacion  $\leftarrow$  m
8:   beneficio  $\leftarrow$  0
9:   while subpoblacion > 0 do
10:    maxBeneficio  $\leftarrow$  -1
11:    fila  $\leftarrow$  -1
12:    for i  $\leftarrow$  0 hasta n do
13:      if indices[i] < n - i then
14:        if poblacion[i][indices[i]] > maxBeneficio then
15:          maxBeneficio  $\leftarrow$  poblacion[i][indices[i]]
16:          fila  $\leftarrow$  i
17:        end if
18:      end if
19:    end for
20:    if elegidos[fila] < 1 then
21:      elegidos[fila]  $\leftarrow$  1
22:      actualizarBeneficio(aux, beneficio, fila)
23:      subpoblacion  $\leftarrow$  subpoblacion - 1
24:    end if
25:    if elegidos[indices[fila]] < 1 then
26:      elegidos[indices[fila]]  $\leftarrow$  1
27:      actualizarBeneficio(aux, beneficio, indices[fila])
28:      subpoblacion  $\leftarrow$  subpoblacion - 1
29:    end if
30:    indices[fila]  $\leftarrow$  indices[fila] + 1
31:  end while
32: end function

```

A continuación unas cuantas aclaraciones sobre este algoritmo. *aux* es una lista de los individuos que se llevan elegidos para facilitar el trabajo a *actualizarBeneficios* sin tener que iterar sobre *elegidos*. En el bucle for de las líneas 12-19 se esconde la decisión voraz, recorreremos (apoyándonos en un control mediante el array *indices*) las parejas que todavía no han sido consideradas buscando la que mayor diversidad genera, es decir la decisión localmente correcta. La ordenación al principio de la función nos permite no tener que recorrer la lista internamente dentro del bucle while, lo que aumentaría mucho la complejidad.

Para posteriores usos en en análisis experimental y adelantando que nuestro algoritmo es una **aproximación** voraz diseñaremos otro algoritmo más costoso que resuelva el

programa de manera exacta usando recursividad y una especie de backtracking:

Algorithm 10 Calcular diversidad de un subconjunto

```

1: function CALCULARDIVERSIDAD(actual)
2:    $total \leftarrow 0$ 
3:   for  $i \leftarrow 0$  hasta  $|actual| - 1$  do
4:     for  $j \leftarrow i + 1$  hasta  $|actual| - 1$  do
5:        $total \leftarrow total + d[actual[i]][actual[j]]$ 
6:        $total \leftarrow total + d[actual[j]][actual[i]]$ 
7:     end for
8:   end for
9:   return  $total$ 
10: end function

```

Algorithm 11 Algoritmo exacto principal

```

1: function RESOLVER(inicio, actual)
2:   if  $|actual| = m$  then
3:      $val \leftarrow calcularDiversidad(actual)$ 
4:     if  $val > mejorBeneficio$  then
5:        $mejorBeneficio \leftarrow val$ 
6:        $mejorSubconjunto \leftarrow actual$ 
7:     end if
8:     return
9:   end if
10:  if  $n - inicio < m - |actual|$  then
11:    return
12:  end if
13:  for  $i \leftarrow inicio$  hasta  $n - 1$  do
14:     $actual.añadir(i)$ 
15:     $resolver(i + 1, actual)$ 
16:     $actual.eliminarÚltimo()$ 
17:  end for
18: end function

```

Ahora, demostraremos que no se cumple la tercera hipótesis del **teorema voraz** mediante un contraejemplo, consideremos la entrada con $n = 5, m = 3$:

```

■ 1
  5 3
  0 100 1 1 1
  100 0 1 1 1
  1 1 0 60 60
  1 1 60 0 60
  1 1 60 60 0

```

En la primera iteración el algoritmo voraz elige el par $(0, 1)$ cuya diversidad aportada es de 200, añade los individuos 0 y 1. En la segunda iteración elige el par $(2, 3)$ cuya

diversidad aportada es 120, añade el individuo 2 esto implica que nos quedamos con la solución **(0, 1, 2)** cuya suma total es de 204. Sin embargo, es fácil ver que eligiendo en triángulo **(2, 3, 4)** como solución la suma total asciende a 360. El problema es que el algoritmo voraz se compromete con el caso localmente óptimo que le impide ya escoger al triángulo globalmente óptimo (una visión que no se contempla). Por tanto acabamos de demostrar que no se cumple la tercera hipótesis del algoritmo voraz: **"sea $S(0, n)$ la secuencia mínima de saltos para ir de 0 a n . Si el primer salto de una solución óptima llega a la posición x_1 , entonces el conjunto de los saltos restantes es la solución óptima del subproblema de ir a n desde x_1 ."**

La pregunta clara es: ¿existe entonces un algoritmo voraz que solucione el problema? En primer lugar destacar que el problema es **NP-hard** así que nuestros algoritmos voraces, en general, no van a poder garantizar optimalidad, aun así cito otros dos argumentos algo más heurísticos:

- **La contribución de un nodo al objetivo depende del contexto:** el aporte de incluir el nodo i cambia según qué otros nodos estén en S . No hay ninguna propiedad intrínseca de i que garantice que sea óptimo incluirlo.
- **Todo criterio local razonable falla:** podríamos intentar ordenar por:
 - Mayor distancia total a todos los demás nodos $\rightarrow$ falla (nodo con gran distancia a uno contamina la suma).
 - Mayor suma de par $\rightarrow$ falla exactamente como demuestra el contraejemplo.

Por tanto solo podemos aspirar a una aproximación voraz, en el análisis experimental demostraremos que la que nosotros hemos escogido es suficientemente óptima.

2.3. Análisis teórico

En este apartado realizaremos un análisis teórico del peor y mejor caso del algoritmo 9.

Empezaremos destacando que consideramos la función ordenar de la línea 3 es $\Theta(n \cdot \log(n))$. Sabiendo que *escogidos* tiene tamaño n y llamando $t_8(n)$ al tiempo de ejecución del algoritmo 8 tenemos que:

$$t_8(n) = n \cdot c_1 + c_2 \implies t_8(n) \in \Theta(n) \quad (2.1)$$

Distingamos ahora cuales son los mejores y peores casos del algoritmo 9. El mejor caso es aquel en el que el subconjunto de m elementos se llena en las primeras $\lceil \frac{m}{2} \rceil$ iteraciones. El peor caso es aquel en el que se recorren muchas parejas repetidas, tantas que se acaban recorriendo todos los pares, como la matriz *poblacion* es n -cuadrada pero despreciamos los elementos nulos de la diagonal tenemos entonces que, quedándonos con la triangular superior, recorreremos $\sum_{i=1}^{n-1} i = \frac{n \cdot (n-1)}{2}$ pares. Teniendo esto en cuenta y denotando por t_M el tiempo del peor caso y por t_m el del mejor caso:

$$\begin{aligned} t_M(n, m) &= n \cdot n \cdot \log(n) + c_1 + \frac{n \cdot (n-1)}{2} \cdot (c_2 + n \cdot (c_3 + c_4) + c_5 + c_6 + c_7) \\ &= n \cdot \log(n) + c_1 + \frac{n \cdot (n-1) \cdot c_8}{2} + \frac{n \cdot (n-1) \cdot (c_3 + c_4)}{2} \end{aligned} \quad (2.2)$$

Entendiendo las c_i como constantes flexibles que agrupan varias líneas de código y $c_8 = c_5 + c_6 + c_7 + c_2$ tenemos que (2.2) $\implies t_M(n, m) \in \Theta(n^3)$. Continuemos con el análisis de t_m

$$\begin{aligned} t_m(n, m) &= n \cdot n \cdot \log(n) + c_1 + \lceil \frac{m}{2} \rceil \cdot (c_2 + n \cdot (c_3 + c_4) + c_5 + c_6 + c_7) \\ &= n \cdot \log(n) + c_1 + \lceil \frac{m}{2} \rceil \cdot c_8 + \lceil \frac{m}{2} \rceil \cdot n \cdot (c_3 + c_4) \end{aligned} \quad (2.3)$$

Como $m \leq n \implies \lceil \frac{m}{2} \rceil \in O(n)$ tenemos que $\lim_{n \rightarrow \infty} \frac{t_m(n, m)}{n^2 \cdot \log(n)} = L \implies t_m(n, m) \in \Theta(n^2 \cdot \log(n))$

Para el algoritmo exacto seremos más escuetos en nuestro análisis ya que no es el protagonista de nuestra memoria y solo lo usamos para justificar nuestra aproximación. Sea t_{10} el tiempo de ejecución del algoritmo 10 y t_{11} el del algoritmo 11:

$$t_{10}(m) = \frac{m(m-1)}{2} \cdot c_1 \implies t_{10}(m) \in \Theta(m^2) \quad (2.4)$$

El algoritmo 2 genera un árbol de recursión, en las $\binom{n}{m}$ hojas tenemos:

$$\begin{aligned} t_{11}(n, m) &= \binom{n}{m} \cdot t_{10}(m) + \binom{n}{m} \cdot c_2 \\ &= \binom{n}{m} \cdot \frac{m(m-1)}{2} \cdot c_1 + \binom{n}{m} \cdot c_2 \\ &= \binom{n}{m} \cdot \left(\frac{c_1 \cdot m(m-1)}{2} + c_2 \right) \end{aligned} \quad (2.5)$$

Agrupando constantes en c_3 :

$$t_{11}(n, m) \in \Theta \left(m^2 \cdot \binom{n}{m} \right) \quad (2.6)$$

2.4. Implementación

- Fichero **AR.cpp**: Implementa la aproximación voraz, comentada y explicada.
- Fichero **AR.h**: Cabecera de la aproximación voraz, para usarla más cómodamente al calcular los tiempos
- Fichero **AR_mooshak.cpp**: Código enviado a Mooshak, pero que no se usa en el estudio estadístico y experimental debido a sus limitaciones.
- Fichero **AR_exacto.cpp**: Implementa un algoritmo para resolver correctamente el problema.
- Fichero **comparador.cpp**: se encarga de generar casos de prueba aleatorios, ejecutar los dos algoritmos anteriores y describir la efectividad de la aproximación con una tasa de fallos, una tasa de error medio en caso de fallo y demás parámetros.

- Fichero **tiempos_AR.cpp**: genera mejores y peores casos, mide, compara sus tamaños y los almacena en un .csv
- Fichero **regresion_AR.py**: a partir de los datos del .csv genera las gráficas de las regresiones lineales.
- Fichero **makefile**: Describe como compilar el proyecto.

Notas: la comparación puede parecer menos extensa que la que hemos hecho en backtracking, pero como solo necesitábamos una tasa de fallos y errores y no sacar conclusiones sobre el tiempo de ejecución hemos decidido simplificarla.

2.5. Envíos

#	Tiempo de Concurso ▾	País	Equipo	Problema	Lenguaje	Resultado	Estado	🔍
416	1404:58:13		g9_g9 08	C_AR	C++	0 <i>Compile Time Error</i>	final	
142	1081:39:23		g9_g9 08	C_AR	C++	2 <i>Accepted</i>	final	

Figura 2.1: Envíos AR

Usuario g9_08:

- Envío **142**: Primer envío, la aproximación fue aceptada a la primera por Mooshak lo cual prácticamente confirma nuestras hipótesis de optimalidad. Este es el envío que hay que **corregir**
- Envío **416**: Segundo envío, intentamos probar el algoritmo exacto en Mooshak pero nos daba error de compilación y encima caímos en que aún arreglando nos iba a lanzar un TLE.

2.6. Análisis experimental

El propósito principal de la sección es demostrar que nuestro algoritmo es lo suficientemente preciso

2.6.1. Optimalidad de la aproximación

A continuación adjuntamos la salida de *comparador.cpp* con los parámetros de prueba por defecto: 1000 test, 10 como valor máximo de n y 0.7 como la máxima relación de $\frac{m}{n}$. Además, hemos limitado el número de fallos que se imprimen por pantalla a 20, aunque aquí no lo pondremos pues no es de relevancia y solo saturaría la vista.

Total pruebas : 1000
Correctas : 613 (61 %)
Fallos : 387 (38 %)
Error medio : 10 %
Error maximo : 43 %

Si bien el porcentaje de éxito de 61 % puede parecer bajo (aunque en realidad es lo esperable para un algoritmo aproximado) lo más relevante es que cuando el programa falla solo comete un 10 % de error medio. Esto deja claro que nuestra aproximación cumple su trabajo, acierta más de la mitad de las veces y en caso de fallar alcanza en promedio un 90 % de eficiencia. Quizá la única parte negativa es que no podemos acotar el error que comete, como podemos comprobar con el pico de 43 % de error máximo, así que aunque a la larga genere buenos resultados en casos concretos puede fallar estrepitosamente. Aún así, teniendo en cuenta además el accepted de Mooshak (que suponemos contará con restricciones en el tiempo de ejecución y en la tasa de error del algoritmo), podemos concluir que el algoritmo aproxima lo suficientemente bien al algoritmo óptimo.

Como vimos en el análisis teórico $t_{11} \in \Theta(m^2 \cdot \binom{n}{m})$ mientras que el peor caso de la aproximación $t_m \in \Theta(n^3)$ es claro que teniendo en cuenta que $m \leq n$ y que $\binom{n}{m}$ crece mucho más rápido que n^3 entonces es claro que prescindir de un $0,38 \cdot 0,1 = 0,038 = 3,8\%$ de precisión a la hora de encontrar la solución óptima compensa por el tiempo que nos ahorramos.

2.7. Contraste

Como tamaño de entrada se ha usado $n_i = 200 \cdot 2^i$ y $m_i = \frac{n_i}{2}$ no necesitamos incidir demasiado en el tamaño de m_i ya que la complejidad depende exclusivamente de n_i , tenemos además que $i \in \{0, 1, 2, 3, 4\}$ ya que el algoritmo tardaba horas en ejecutarse para valores mayores. Además, hemos tomado, como se sugería en teoría, 5 medidas diferentes para cada i con el fin de calcular su mediana y evitar el ruido que se pueda generar. A continuación se adjuntan las medidas tomadas y los ajustes que avalan nuestros cálculos teóricos.

Longitud	Peor Caso	Mejor Caso
200	3,92324	1,13188
400	14,2588	3,5389
800	117,544	16,501
1600	1093,95	76,1616
3200	18723,5	354,473

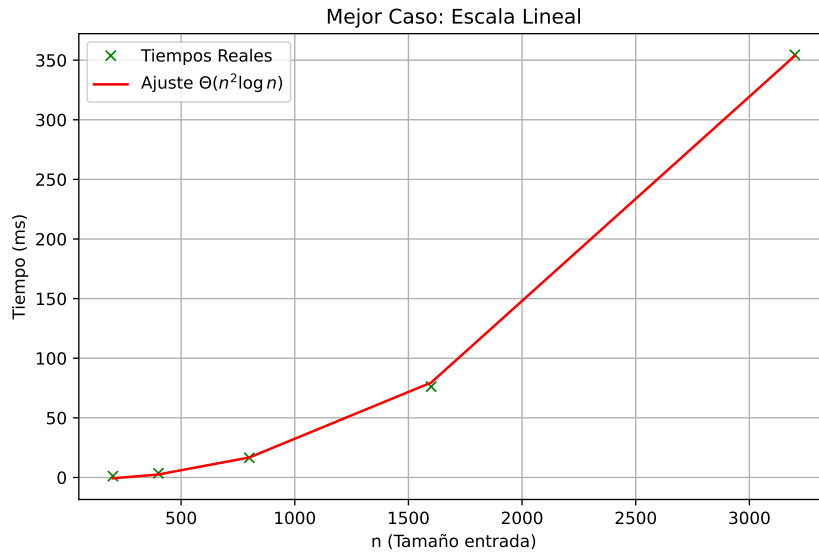


Figura 2.2: Ajuste mejor caso, $R^2 = 0,9998$

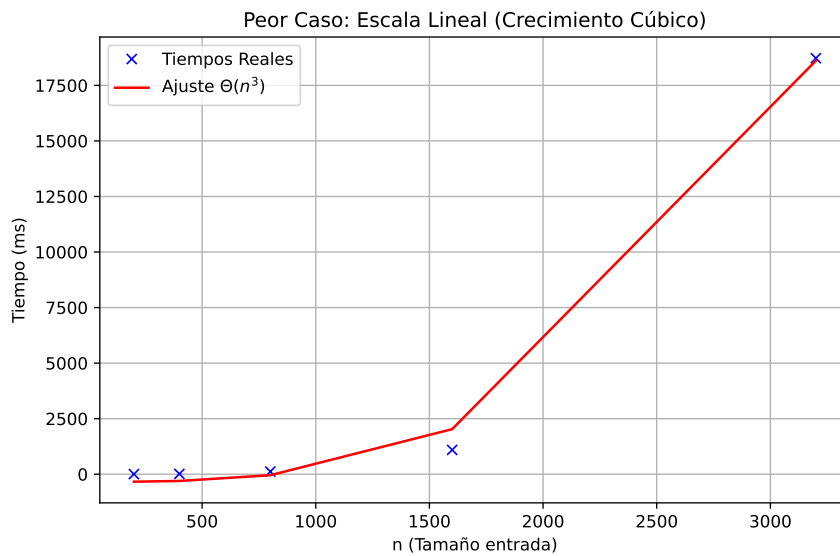


Figura 2.3: Ajuste peor caso, $R^2 = 0,9959$

La estrategia para generar el peor caso real $\Theta(n^3)$ consiste en diseñar una matriz de distancias basada en un orden lexicográfico decreciente $BASE - j \cdot n - i$ que engaña al algoritmo voraz, obligándolo a saltar constantemente entre filas y a procesar una enorme cantidad de iteraciones inútiles

Capítulo 3

Uso de la IA

Se ha usado Claude para generar en gran parte el código de *comparador.cpp* para comparar las soluciones exactas y las soluciones voraces aproximadas del problema de AR. Se le pasaron ambos códigos, un formato de salida deseado y el código de algunos test oráculos usados en otros ejercicios y se le pidió que generara código que en base a distintos casos de prueba generados aleatoriamente midiera como de bien aproxima la solución el algoritmo voraz mediante una tasa de aciertos y una tasa media de error en caso de fallo.

También se ha usado para identificar el árbol de recursión en el análisis teórico del algoritmo 1, se le pasó el código de dicho algoritmo y se le solicitó algo de orientación para realizar el análisis.

Además, nos ha ayudado a como generar casos de prueba del peor caso de AR, pues como se puede apreciar en nuestra explicación era bastante complicado. Simplemente le pasamos el análisis teórico y el código del programa *tiempos_AR.cpp* y le pedimos un método para generar casos que forzarán el recorrido de todos los pares.

Por último, se ha usado en *tiempos_BT.cpp* para el tema del formato de impresión en el csv y en el cout, en un principio nos quedaba bastante sucio e incluso *grafica_poda.py* fallaba por errores en la redacción del csv, además, el código de *tiempos_BT.cpp* quedaba bastante desordenado. Así que le pasamos el código, las salidas pertinentes y le pedimos que dejara un formato más limpio para mostrarlos.

Capítulo 4

Conclusiones

El mayor aporte de esta práctica ha sido el estudio estadístico que hemos tenido que hacer para demostrar que nuestros algoritmos eran eficientes. También resaltar el análisis teórico que no era nada trivial y, al no ser mecánico como en la práctica anterior (uso del Teorema Maestro) ha sido otro reto.

Consideramos que ahorraría mucho trabajo que en AR los problemas se pudieran resolver con un algoritmo voraz óptimo, aunque entendemos la intención de mostrar que a veces vale la pena sacrificar algo de precisión a favor de una ejecución mucho más rápida.

4.1. IMPORTANTE

Algunas notas que consideramos importantes comentar aquí. Hemos adjuntado el código crudo enviado a Mooshak pues necesitábamos un main para que lo aceptara, pero no ha sido el código programado desde un principio ni el usado en las experimentaciones, el formato cabecera-implementación es el que hemos seguido desde un principio para facilitar su uso, así que es además el código comentado. Rogamos comprensión y esperamos que no sea de importancia.

En backtracking no hemos incluido una sección de contraste, pues no había nada que contrastar. Es imposible construir peores casos teóricamente no podemos predecir como se va a comportar la poda en tamaños considerables, además, para el algoritmo con FO ni siquiera hemos podido extraer un orden exacto. Simplemente demostramos que la poda FO es mejor que la poda no FO. Más aún, la complejidad depende dos parámetros, tendríamos que hacer gráficas en 3D. Esperemos que todo esto justifique nuestra decisión.